DAM | DISEÑO DE INTERFACES | UNIDAD 3

CheatSheet #11: Ejercicio 5 · Gestión Empresarial

WPF · Ventana personalizada · Panel principal · Alta de empleados con SQLite

IES Henri Matisse
2º DAM · Repaso práctico

EJERCICIO 05 Aplicación de gestión empresarial — WPF + C#

Objetivo: construir una ventana de escritorio sin marco estándar, organizar una pantalla de gestión y permitir altas de empleados. El diálogo escribe los datos en una base SQLite local.

Flujo: se inicia `MainWindow` → su constructor crea/verifica `empresa.db` → el usuario abre "Nuevo empleado" → se validan los campos → un `INSERT` parametrizado guarda el empleado.

1. Ventana sin marco

`WindowStyle="None"` elimina el marco nativo; `AllowsTransparency` permite el fondo transparente y el `Border` dibuja las esquinas.

2. Distribución

Un `Grid` divide encabezado y cuerpo; el cuerpo separa navegación lateral del panel mediante columnas.

3. Acciones de ventana

Los botones programan minimizar, alternar maximizado/restaurado y cerrar; el encabezado permite arrastrar.

VENTANA Y DISPOSICIÓN · XAML — MAINWINDOW.XAML

```xaml
<Window
    WindowStyle="None"
    ResizeMode="NoResize"
    AllowsTransparency="True"
    Background="Transparent"
    MouseLeftButtonDown="Window_MouseLeftButtonDown">
  <Border CornerRadius="20">
    <Grid>
      <Grid.RowDefinitions>
        <RowDefinition Height="70" /> <!-- barra -->
        <RowDefinition Height="*" /> <!-- contenido -->
      </Grid.RowDefinitions>
      <!-- Fila 1: menu lateral + panel central -->
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="220" />
        <ColumnDefinition Width="*" />
      </Grid.ColumnDefinitions>
    </Grid>
  </Border>
</Window>
```

ACCIONES DEL ENCABEZADO — C#

```csharp
private void Window_MouseLeftButtonDown(
    object sender, MouseButtonEventArgs e)
{
    if (e.LeftButton == MouseButtonState.Pressed)
        DragMove();
}

private void BtnMaximizar_Click(
    object sender, RoutedEventArgs e)
{
    WindowState = WindowState == WindowState.Maximized
        ? WindowState.Normal
        : WindowState.Maximized;
}

// Minimizar: WindowState.Minimized
// Cerrar: Close()
```

DETALLES QUE CONVIENE RECORDAR

- ! Al quitar el marco nativo hay que proporcionar controles propios para cerrar y minimizar.
- ! `ResizeMode="NoResize"` impide redimensionar arrastrando el borde.
- ! `DragMove()` se llama mientras el botón izquierdo está pulsado.
- ! Las tarjetas "Personal", "Activos" y "Bajas" muestran valores escritos en XAML, no consultas a SQLite.

SQLite y alta de empleados

Inicialización idempotente · Ventana modal propietaria · Validación · INSERT parametrizado

IES Henri Matisse
Parte funcional del ejercicio

PERSISTENCIA Crear la tabla y guardar empleados — Microsoft.Data.Sqlite

Esquema: `Empleados` guarda identificador, nombre, apellidos, departamento, puesto y fecha de alta. La ruta se calcula desde `AppDomain.CurrentDomain.BaseDirectory`, es decir, junto a la aplicación compilada.

INICIALIZACIÓN DE BASE Y TABLA — C# · SQLITE

```
private const string DatabaseName = "empresa.db";

public static void EnsureDatabaseExists()
{
    var path = Path.Combine(
        AppDomain.CurrentDomain.BaseDirectory, DatabaseName);
    using var connection = new SqliteConnection(
        $"Data Source={path}");
    connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText = @"
      CREATE TABLE IF NOT EXISTS Empleados (
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Nombre TEXT NOT NULL,
        Apellidos TEXT NOT NULL,
        Departamento TEXT NOT NULL,
        Puesto TEXT NOT NULL,
        FechaAlta TEXT NOT NULL DEFAULT (date('now'))
      );";
    command.ExecuteNonQuery();
}
```

CONSTRUCCIÓN Y APERTURA MODAL — C# · WPF

```
public MainWindow()
{
    InitializeComponent();
    EnsureDatabaseExists();
}

private void BtnNuevoEmpleado_Click(
    object sender, RoutedEventArgs e)
{
    var dialog = new AddEmpleadoDialog();
    dialog.Owner = this;
    dialog.ShowDialog();
}
```

El diálogo se construye en C#; sus TextBox, ComboBox y botones se crean por código. `Owner` y `ShowDialog()` lo vinculan a la ventana principal y lo muestran de forma modal.

VALIDACIÓN Y GUARDADO SEGURO — C# · SQL PARAMETRIZADO

```
var nombre = nombreTextBox.Text.Trim();
var apellidos = apellidosTextBox.Text.Trim();
var departamento = departamentoCombo.SelectedItem?.ToString();
var puesto = puestoCombo.SelectedItem?.ToString();
if (string.IsNullOrWhiteSpace(nombre) || string.IsNullOrWhiteSpace(apellidos) ||
    string.IsNullOrWhiteSpace(departamento) || string.IsNullOrWhiteSpace(puesto))
{
    MessageBox.Show("Debes completar todos los campos...");
    return;
}
command.CommandText = @"INSERT INTO Empleados (Nombre, Apellidos, Departamento, Puesto)
    VALUES (@nombre, @apellidos, @departamento, @puesto);";
command.Parameters.AddWithValue("@nombre", nombre);
command.Parameters.AddWithValue("@apellidos", apellidos);
command.Parameters.AddWithValue("@departamento", departamento);
command.Parameters.AddWithValue("@puesto", puesto);
command.ExecuteNonQuery();
```

TRAMPAS Y LÍMITES DE LA IMPLEMENTACIÓN

- ! `CREATE TABLE IF NOT EXISTS` permite iniciar la app varias veces sin recrear la tabla.
- ! El diálogo valida los cuatro datos obligatorios y muestra errores/excepciones con `MessageBox`.
- ! Los parámetros SQL separan los valores del comando y evitan concatenar entrada del usuario en la consulta.
- ! El alta sí persiste en SQLite, pero la lista “Últimos empleados” todavía usa elementos estáticos y no se recarga desde la tabla.